Nainisha Bandari

Nainishabandari1111@gmail.com

Database Optimization

Objective

The goal of this task was to improve:

- Database structure
- Query performance
- Schema organization
- Relationships between collections
- MongoDB indexing

Optimized Schemas

The following schemas were reviewed and optimized:

Schema		Improvements
User		Email indexing, createdAt indexing
Product		Category indexing, timestamps, cleaner image structure
TryOn		userId indexing, createdAt indexing
Profile		userId indexing, timestamps
Wardrobe		category indexing, userId indexing

Image		imageType indexing, timestamps
Cart		Optimized relationship structure
Order		Added efficient references and indexing
Measurement		Added user-based relationship

Database Optimization Improvements

1. Added Indexes

Indexes were added to improve query performance.

Added Indexes

- email
- category
- createdAt
- orderStatus
- userId

Example

```
productSchema.index({ category: 1 });
productSchema.index({ createdAt: -1 });
```

2. Improved Relationships

Instead of storing repeated full objects, only references are stored.

Example

```
userId: {
  type: mongoose.Schema.Types.ObjectId,
  ref: "User"
}
```

Benefits:

- Reduced duplicate data
- Smaller database size
- Faster queries
- Better scalability

3. Product Schema Optimization

Changes made:

- Added category indexing
- Added timestamps
- Simplified image array structure
- Improved schema organization

Optimized Structure

images: [String]

4. Query Performance Optimization

Used:

- .select()
- .lean()

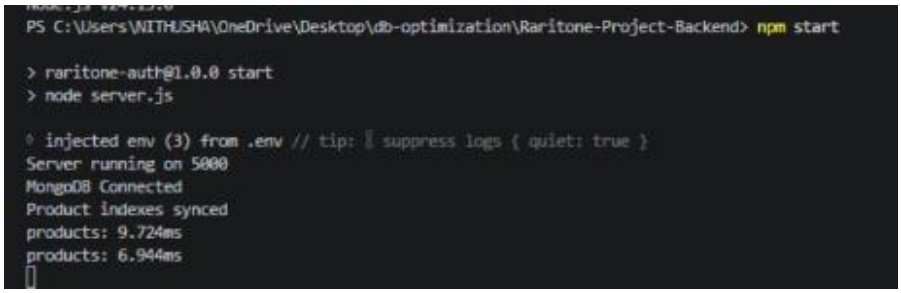
to improve query efficiency.

Example

```
const users = await User.find()
  .select("name email")
  .lean();
```

Benefits:

- Faster response time
- Reduced memory usage
- Smaller payload size.



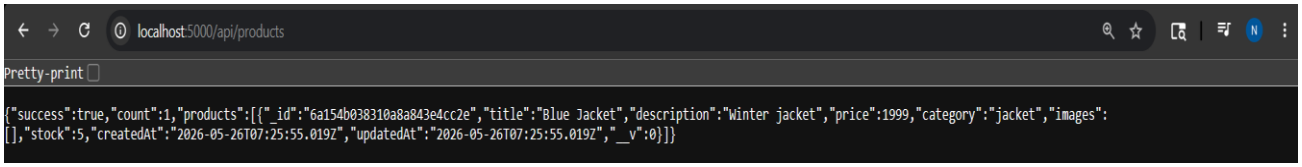
5. Timestamp Optimization

Added automatic timestamps using:

timestamps: true

Automatically creates:

- createdAt
- updatedAt



6. MongoDB Configuration Optimization

Added:

```
mongoose.set("strictQuery", true);
```

Benefits:

- Safer queries
- Better query handling
- Improved database consistency

Verification Process

- The following methods were used to verify optimization:

Checked Indexes

`db.products.getIndexes()`

- **Checked Query Performance**

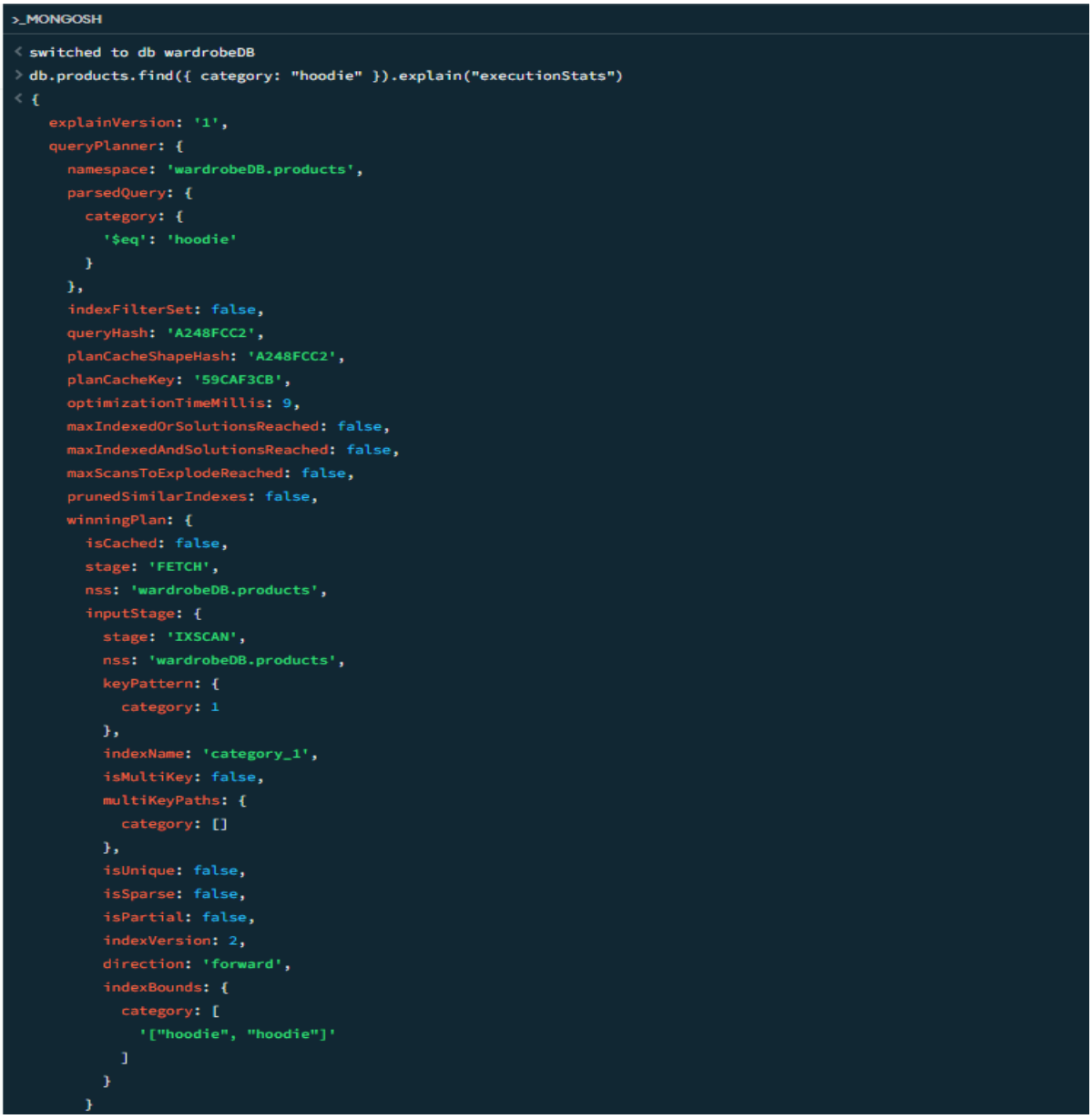
  `db.products.find({ category: "Men" }).explain("executionStats")`

- Verified:

  `IXSCAN`

- Instead of:

  `COLLSCAN`



```
>_MONGOSH
< switched to db wardrobeDB
> db.products.find({ category: "hoodie" }).explain("executionStats")
< {
    explainVersion: '1',
    queryPlanner: {
      namespace: 'wardrobeDB.products',
      parsedQuery: {
        category: {
          '$eq': 'hoodie'
        }
      },
      indexFilterSet: false,
      queryHash: 'A248FCC2',
      planCacheShapeHash: 'A248FCC2',
      planCacheKey: '59CAF3CB',
      optimizationTimeMillis: 9,
      maxIndexedOrSolutionsReached: false,
      maxIndexedAndSolutionsReached: false,
      maxScansToExplodeReached: false,
      prunedSimilarIndexes: false,
      winningPlan: {
        isCached: false,
        stage: 'FETCH',
        nss: 'wardrobeDB.products',
        inputStage: {
          stage: 'IXSCAN',
          nss: 'wardrobeDB.products',
          keyPattern: {
            category: 1
          },
          indexName: 'category_1',
          isMultiKey: false,
          multiKeyPaths: {
            category: []
          },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'forward',
          indexBounds: {
            category: [
              '["hoodie", "hoodie"]'
            ]
          }
        }
      }
```

Final Outcome

- The database optimization resulted in:
- Faster database queries
- Better indexing
- Cleaner schema structure
- Reduced duplicate data
- Improved scalability
- Better MongoDB performance
- Optimized relationships
- Cleaner data organization